Refactor proyecto Market



Oihane Zubiaurre
Oiher Bernet
Julen Haro
Ingeniería informática UPV/EHU
Github: https://github.com/Oihane870/Market_IS2

1. Oihane Zubiaurre 2
Refactorización "Write short units of code" (initializeBD) 2
Refactorización "Write simple units of code" (resolveErreklamazioa) 4
Refactorización “Duplicate code” (SalaketaGUI) 4
Refactorización "Keep unit interfaces small" (addSale, Seller) 4
2. Oiher Bernet 4
Refactorización: "Write short units of code" (buyBasket) 4
Refactorización: "Write simple units of code" (addToBasket) 4
Refactorización: "Duplicate code" (isAdmin y isLogged) 4
Refactorizacion: “Keep unit interfaces small” (createSale) 4
3. Julen Haro 4
Refactorización: "Write short units of code" (buyProduct) 4
Refactorización: "Write simple units of code" (resolveReport) 4
Refactorización: "Duplicate code" (getSales y getPublishedSales) 4
Refactorización: “Keep unit interfaces small” (constructor de Sale) 4

1. Oihane Zubiaurre

Refactorización "Write short units of code" (initializeBD)

1. Código inicial

```
public void initialiseDB(){
    db.getTransaction().begin();
    try {
        //Create sellers
        Seller seller1=new Seller("seller1@gmail.com","Aitor Fernandes","aurrera");
        Seller seller2=new Seller("seller22@gmail.com","Ane Gastañaga","aurrera");
        Seller seller3=new Seller("seller3@gmail.com","Test Seller","aurrera");

        Admin admin = new Admin("admin@gmail.com","Admin","admin123");
        db.persist(admin);

        //Create products
        Date today = UtilDate.trim(new Date());

        seller1.addSale("futbol baloia", "oso polita, gutxi erabilita", 2, 10, today, null);
        seller1.addSale("salomon mendiko botak", "44 zenbakia, 3 ateraldi",2,  20, today, null);
        seller1.addSale("samsung 42\" telebista", "berria, erabili gabe", 1, 175, today, null);

        seller2.addSale("imac 27", "7 urte, dena ondo dabil", 1, 200,today, null);
        seller2.addSale("iphone 17", "oso gutxi erabilita", 2, 400, today, null);
        seller2.addSale("orbea mendiko bisikleta", "29\" 10 urte, mantenua behar du", 3,225, today, null);
        seller2.addSale("polar kilor erlojua", "Vantage M, ondo dago", 3, 30, today, null);

        seller3.addSale("sukaldeko mahaia", "1.8*0.8, 4 aulkiekin. Presio finkoa", 3,45, today, null);

        db.persist(seller1);
        db.persist(seller2);
        db.persist(seller3);

        db.getTransaction().commit();
        System.out.println("Db initialised");
    }
    catch (Exception e){
        e.printStackTrace();
    }
}
```

2. Código refactorizado

```
public void initializeDB(){
    db.getTransaction().begin();
    try {
        //Create sellers
        Seller seller1=new Seller("seller1@gmail.com","Aitor Fernandez","aurrera");
        Seller seller2=new Seller("seller22@gmail.com","Ane Gaztañaga","aurrera");
        Seller seller3=new Seller("seller3@gmail.com","Test Seller","aurrera");

        Admin admin = new Admin("admin@gmail.com","Admin","admin123");
        db.persist(admin);

        //Create products
        Date today = UtilDate.trim(new Date());

        createInitialSales(seller1, seller2, seller3, today);

        persistSellers(seller1, seller2, seller3);

        db.getTransaction().commit();
        System.out.println("Db initialized");
    }
    catch (Exception e){
        e.printStackTrace();
    }
}

private void createInitialSales(Seller seller1, Seller seller2, Seller seller3, Date today) {
    seller1.addSale("futbol baloia", "oso polita, gutxi erabilita", 2, 10, today, null);
    seller1.addSale("salomon mendiko botak", "44 zenbakia, 3 ateraldi",2,  20, today, null);
    seller1.addSale("samsung 42\" telebista", "berria, erabili gabe", 1, 175, today, null);

    seller2.addSale("imac 27", "7 urte, dena ondo dabil", 1, 200,today, null);
    seller2.addSale("iphone 17", "oso gutxi erabilita", 2, 400, today, null);
    seller2.addSale("orbea mendiko bizikleta", "29\" 10 urte, mantenua behar du", 3,225, today, null);
    seller2.addSale("polar kilor erlojua", "Vantage M, ondo dago", 3, 30, today, null);

    seller3.addSale("sukaldeko mahaia", "1.8*0.8, 4 aulkiekin. Prezio finkoa", 3,45, today, null);
}

private void persistSellers(Seller seller1, Seller seller2, Seller seller3) {
    db.persist(seller1);
    db.persist(seller2);
    db.persist(seller3);
}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code smell: Se detecto un método con 21 líneas de código. Este método creaba Sellers les añadía sales y después los metía en la base de datos. Tras realizar la refactorización ese método se dividió en 3 y quedaron 12 líneas en el método principal, 8 en createInitialSales y 3 en persistSellers.

Refactorización: Para hacer el código más legible se crearon dos nuevos métodos (Refactor > Extract Metod) de esa manera tenemos un método que inicializa las Sales de cada Seller y otro método para meter los Sellers en la base de datos.

4. Miembro que ha realizado la refactorización
    - Realizado por: Oihane Zubiaurre

Refactorización "Write simple units of code" (resolveErreklamazioa)

1. Código inicial

```
public boolean resolveErreklamazioa(Integer saleNumber, boolean accept) {
    try {
        db.getTransaction().begin();
        Sale sale = db.find(Sale.class, saleNumber);
        if (sale == null || sale.getErreklamazioa() == null) {
            db.getTransaction().rollback(); return false;
        }

        if (accept) {
            Seller buyer = sale.getBuyer();
            Seller seller = sale.getSeller();

            if (buyer != null && seller != null) {
                buyer.addMoney(sale.getPrice());
                seller.addMoney(-sale.getPrice());


                Mugimenduak mBuyer = new Mugimenduak("ITZULKETA", new java.util.Date(), buyer);
                mBuyer.setSale(sale);
                db.persist(mBuyer);

                Mugimenduak mSeller = new Mugimenduak("ITZULKETA_KENTZEA", new java.util.Date(), seller);
                mSeller.setSale(sale);
                db.persist(mSeller);


                buyer.getPurchasedSales().remove(sale);
                sale.setBuyer(null);

                db.merge(buyer);
                db.merge(seller);
            }
        }
```

2. Código refactorizado

```
public boolean resolveErreklamazioa(Integer saleNumber, boolean accept) {
    try {
        db.getTransaction().begin();
        Sale sale = db.find(Sale.class, saleNumber);
        if (sale == null || sale.getErreklamazioa() == null) {
            db.getTransaction().rollback(); return false;
        }

        handleErreklamazioaAcceptance(accept, sale);

        closeErreklamazioa(sale);

        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}

private void closeErreklamazioa(Sale sale) {
    Erreklamazioa err = sale.getErreklamazioa();
    sale.setErreklamazioa(null);
    db.merge(sale);

    if (err != null) {
        Erreklamazioa dbErr = db.find(Erreklamazioa.class, err.getId());
        if (dbErr != null) err.setTratatuta(true);
        db.merge(err);
    }
}

private void handleErreklamazioaAcceptance(boolean accept, Sale sale) {
    if (accept) {
        Seller buyer = sale.getBuyer();
        Seller seller = sale.getSeller();

        if (buyer != null && seller != null) {
            buyer.addMoney(sale.getPrice());
            seller.addMoney(-sale.getPrice());


            Mugimenduak mBuyer = new Mugimenduak("ITZULKETA", new java.util.Date(), buyer);
            mBuyer.setSale(sale);
            db.persist(mBuyer);

            Mugimenduak mSeller = new Mugimenduak("ITZULKETA_KENTZEA", new java.util.Date(), seller);
            mSeller.setSale(sale);
            db.persist(mSeller);


            buyer.getPurchasedSales().remove(sale);
            sale.setBuyer(null);

            db.merge(buyer);
            db.merge(seller);
        }
    }
}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Bad Smell: El método tenía complejidad 9, para mantener el código simple debería tener un máximo de complejidad 5. Para bajar la complejidad del método se crearon otros dos métodos, handleErreklamazioaAcceptance de complejidad 4 y closeErreklamazioa de complejidad 3. De esa manera la complejidad del método principal bajó a 4.

Refactorización: Se crearon dos nuevos métodos (Refactor > Extract Method). En handleErreklamazioaAcceptance se comprueba el boolean accept y en base a eso

se empieza el proceso de devolución del producto y en closeErreklamazioa se cierra la reclamación una vez ha sido tratada en el método anterior.

4. Miembro que ha realizado la refactorización
    - Realizado por: Oihane Zubiaurre

Refactorización “Duplicate code” (SalaketaGUI)

1. Código inicial

```
public SalaketaGUI(String userEmail,Sale sale) {
    this.sale = sale;
    this.userEmail = userEmail;

    setTitle(ResourceBundle.getBundle("Etiquetas").getString("CreateSaleGUI.Title"));
    setSize(400, 300);
    setLayout(null);
    setLocationRelativeTo(null);

    JLabel label = new JLabel(ResourceBundle.getBundle("Etiquetas").getString("Reason"));
    label.setBounds(30, 20, 100, 20);
    add(label);

    textArea = new JTextArea();
    JScrollPane scroll = new JScrollPane(textArea);
    scroll.setBounds(30, 50, 320, 120);
    add(scroll);

    btnEnviar = new JButton(ResourceBundle.getBundle("Etiquetas").getString("SalaketaGUI.Bidali"));
    btnEnviar.setBounds(60, 200, 100, 30);
    add(btnEnviar);

    btnCerrar = new JButton(ResourceBundle.getBundle("Etiquetas").getString("Close"));
    btnCerrar.setBounds(200, 200, 100, 30);
    add(btnCerrar);
```

2. Código refactorizado

```
private static final ResourceBundle BUNDLE = ResourceBundle.getBundle("Etiquetas");


public SalaketaGUI(String userEmail,Sale sale) {
    this.sale = sale;
    this.userEmail = userEmail;

    setTitle(BUNDLE.getString("CreateSaleGUI.Title"));
    setSize(400, 300);
    setLayout(null);
    setLocationRelativeTo(null);

    JLabel label = new JLabel(BUNDLE.getString("Reason"));
    label.setBounds(30, 20, 100, 20);
    add(label);

    textArea = new JTextArea();
    JScrollPane scroll = new JScrollPane(textArea);
    scroll.setBounds(30, 50, 320, 120);
    add(scroll);

    btnEnviar = new JButton(BUNDLE.getString("SalaketaGUI.Bidali"));
    btnEnviar.setBounds(60, 200, 100, 30);
    add(btnEnviar);

    btnCerrar = new JButton(BUNDLE.getString("Close"));
    btnCerrar.setBounds(200, 200, 100, 30);
    add(btnCerrar);
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code smell: Había varias duplicaciones de ResourceBundle.getBundle(“Etiquetas”).

Refactorización: La solución ha sido llamar a ese método una sola vez y recoger su valor en una variable para simplificar el código y no estar llamando a la misma función varias veces.

4. Miembro que ha realizado la refactorización
    - Realizado por: Oihane Zubiaurre

Refactorización "Keep unit interfaces small" (addSale, Seller)

1. Código inicial

```
public Sale addSale(String title, String description, int status, float price,  Date pubDate, File file)  {
    Sale sale=new Sale(new ProductInfo(title, description, status, price), pubDate, file, this);
    sales.add(sale);
    return sale;
}
```

2. Código refactorizado

```
public Sale addSale(ProductInfo productInfo, Date pubDate, File file)  {
    Sale sale=new Sale(productInfo, pubDate, file, this);
    sales.add(sale);
    return sale;
}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Bad Smell: El código tenía 6 parámetros cuando debería tener como máximo 4 para mejorar la mantenibilidad. Tras utilizar la clase ProductInfo los parámetros del metodo bajaron a 3.

Refactorización: Aprovechando que ya existe una clase de ProductInfo donde agrupa title, description, status y price se hizo una refactorización de addSale(Refactor > Change Method Signature). Se cambió también la manera en la que se llamaba a ese método en todas las clases, tanto en dataAccess como en las clases de test.

4. Miembro que ha realizado la refactorización
    - Realizado por: Oihane Zubiaurre

2. Oiher Bernet

Refactorización: "Write short units of code" (buyBasket)

1. Código inicial

```
public boolean buyBasket(String buyerEmail) {
    try {
        db.getTransaction().begin();
        Seller buyer = db.find(Seller.class, buyerEmail);
        if (buyer == null) {
            db.getTransaction().rollback();
            return false;
        }

        List<Sale> basket = buyer.getBasket();

        if (basket == null || basket.isEmpty()) {
            db.getTransaction().rollback();
            return false;
        }

        Seller seller = basket.get(0).getSeller();

        float total = 0;

        for (Sale s : basket) {
            if (!s.getSeller().equals(seller)) {
                db.getTransaction().rollback();
                return false;
            }
            total += s.getPrice();
```

```
        }

        if (buyer.getMoney() < total) {
            db.getTransaction().rollback();
            return false;
        }

        buyer.addMoney(-total);

        for (Sale s : basket) {
            s.setBuyer(buyer);
            buyer.addPurchasedSale(s);

            Bidalketa bidalketa = new Bidalketa(s);
            s.setBidalketa(bidalketa);
            db.persist(bidalketa);

            Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new
java.util.Date(), buyer);
            mugimendu.setSale(s);
            db.persist(mugimendu);

            db.merge(s);
        }

        basket.clear();

        db.merge(buyer);
        db.merge(seller);
```

```
            db.getTransaction().commit();
            return true;

        } catch (Exception e) {
            e.printStackTrace();
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }
            return false;
        }
    }
```

2. Código refactorizado

```
    public boolean buyBasket(String buyerEmail) {
        try {
            db.getTransaction().begin();
            if (!processBasketPurchase(buyerEmail)) {
                db.getTransaction().rollback();
                return false;
            }
            db.getTransaction().commit();
            return true;
        } catch (Exception e) {
            e.printStackTrace();
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }
            return false;
        }
```

```
}

private boolean processBasketPurchase(String buyerEmail) {
    Seller buyer = db.find(Seller.class, buyerEmail);
    List<Sale> basket = buyer != null ? buyer.getBasket() : null;
    if (!isValidBasket(buyer, basket)) return false;

    float total = calculateBasketTotal(basket);
    if (buyer.getMoney() < total) return false;

    executeBasketCheckout(buyer, basket, total);
    return true;
}

private boolean isValidBasket(Seller buyer, List<Sale> basket) {
    if (buyer == null || basket == null || basket.isEmpty()) return false;
    Seller firstSeller = basket.get(0).getSeller();
    for (Sale s : basket) {
        if (!s.getSeller().equals(firstSeller)) return false;
    }
    return true;
}

private float calculateBasketTotal(List<Sale> basket) {
    float total = 0;
    for (Sale s : basket) {
        total += s.getPrice();
    }
    return total;
```

```
}

private void executeBasketCheckout(Seller buyer, List<Sale> basket, float total) {
    buyer.addMoney(-total);
    Seller seller = basket.get(0).getSeller();
    for (Sale s : basket) {
        processSingleSale(buyer, s);
    }
    basket.clear();
    db.merge(buyer);
    db.merge(seller);
}

private void processSingleSale(Seller buyer, Sale s) {
    s.setBuyer(buyer);
    buyer.addPurchasedSale(s);
    Bidalketa bidalketa = new Bidalketa(s);
    s.setBidalketa(bidalketa);
    db.persist(bidalketa);
    Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new
java.util.Date(), buyer);
    mugimendu.setSale(s);
    db.persist(mugimendu);
    db.merge(s);
}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: buyBasket es muy largo y ya que tiene más de 15 líneas por tanto tenemos que hacerlo más pequeño generando funciones para utilizarlos y poder hacerlo la función que queremos en este caso buyBasket con menos de 15 líneas.

Refactorización: Se ha generado las funciones auxiliares mediante pinchando lo que queríamos convertir en función y poniendo Refactor => Extract Method.

4. Miembro que ha realizado la refactorización
    - Realizado por: Oiher Bernet

Refactorización: "Write simple units of code" (addToBasket)

1. Código inicial

```
public boolean addToBasket(String buyerEmail, Integer saleNumber) {
   try {
      db.getTransaction().begin();
      Seller buyer = db.find(Seller.class, buyerEmail);
      Sale sale = db.find(Sale.class, saleNumber);

      if (buyer == null || sale == null || sale.getBuyer() != null) {
         db.getTransaction().rollback();
         return false;
      }

      List<Sale> basket = buyer.getBasket();

      for (Sale s : basket) {
         if (s.getSaleNumber().equals(saleNumber)) {
            db.getTransaction().rollback();
            return false;
         }
      }

      if (basket.isEmpty()) {
         basket.add(sale);
```

```
            db.merge(buyer);
            db.getTransaction().commit();
            return true;
        }

        Seller firstSeller = buyer.getBasket().get(0).getSeller();

        if (!sale.getSeller().equals(firstSeller)) {
            return false;
        }

        buyer.addToBasket(sale);
        db.merge(buyer);

        db.getTransaction().commit();
        return true;

    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}
```

2. Código refactorizado

```
public boolean addToBasket(String buyerEmail, Integer saleNumber) {
    try {
        db.getTransaction().begin();
        boolean success = processAddToBasket(buyerEmail, saleNumber);
        if (!success) {
```

```
            db.getTransaction().rollback();
            return false;
        }
        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
}

private boolean processAddToBasket(String buyerEmail, Integer saleNumber) {
    Seller buyer = db.find(Seller.class, buyerEmail);
    Sale sale = db.find(Sale.class, saleNumber);

    if (!isValidSaleForBasket(buyer, sale)) return false;

    List<Sale> basket = buyer.getBasket();
    if (isSaleAlreadyInBasket(basket, saleNumber)) return false;

    if (basket.isEmpty()) { // 1 punto (if)
        return handleEmptyBasket(buyer, sale);
    }

    return handleNonEmptyBasket(buyer, sale, basket);
}

private boolean isValidSaleForBasket(Seller buyer, Sale sale) {
```

```
    if (buyer == null) return false;
    if (sale == null) return false;
    if (sale.getBuyer() != null) return false;
    return true;
}

private boolean isSaleAlreadyInBasket(List<Sale> basket, Integer saleNumber) {
    for (Sale s : basket) {
        if (s.getSaleNumber().equals(saleNumber)) {
            return true;
        }
    }
    return false;
}

private boolean handleEmptyBasket(Seller buyer, Sale sale) {
    buyer.getBasket().add(sale);
    db.merge(buyer);
    return true;
}

private boolean handleNonEmptyBasket(Seller buyer, Sale sale, List<Sale> basket)
{
    Seller firstSeller = basket.get(0).getSeller();
    if (!sale.getSeller().equals(firstSeller)) {
        return false;
    }
    buyer.addToBasket(sale);
    db.merge(buyer);
```

```
    return true;
}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: addToBasket la complejidad ciclomática de la función es 9 pero no queremos que la complejidad ciclomática sea major de 4 por tanto como en el bad smell anterior separamos diferentes funciones en el que la complejidad ciclomática sea igual o menos que 4.

Refactorización: Se ha generado las funciones auxiliares mediante pinchando lo que queríamos convertir en función y poniendo Refactor => Extract Method.

4. Miembro que ha realizado la refactorización
    - Realizado por: Oiher Bernet

Refactorización: "Duplicate code" (isAdmin y isLogged)

1. Código inicial

```
@WebMethod public Seller isLogged(String log, String pass) {
TypedQuery<Seller> query = db.createQuery("SELECT s FROM Seller s WHERE
s.name=?1 AND s.pass=?2",Seller.class);
query.setParameter(1, log);
query.setParameter(2, pass);
if(!query.getResultList().isEmpty()) //User Existitzen bada
return query.getResultList().get(0);
else return null;
}
```

```
public Admin isAdmin(String log, String pass) {
TypedQuery<Admin> query = db.createQuery(
"SELECT a FROM Admin a WHERE a.name=?1 AND a.pass=?2",
Admin.class
);
query.setParameter(1, log);
query.setParameter(2, pass);

if (!query.getResultList().isEmpty())
return query.getResultList().get(0);
```

```
else
return null;
}
```

2. Código refactorizado

```
private <T> T getSingleResultOrNull(TypedQuery<T> query) {
    List<T> results = query.getResultList();
    return results.isEmpty() ? null : results.get(0);
}

@WebMethod
public Seller isLogged(String log, String pass) {
    TypedQuery<Seller> query = db.createQuery("SELECT s FROM Seller s
WHERE s.name=?1 AND s.pass=?2", Seller.class);
    query.setParameter(1, log);
    query.setParameter(2, pass);
    return getSingleResultOrNull(query);
}

public Admin isAdmin(String log, String pass) {
    TypedQuery<Admin> query = db.createQuery(
        "SELECT a FROM Admin a WHERE a.name=?1 AND a.pass=?2",
        Admin.class
    );
    query.setParameter(1, log);
    query.setParameter(2, pass);
    return getSingleResultOrNull(query);
}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: isAdmin y isLogged tienen codigo en comun que se puede generar con una funcion como hemos hecho en los anterior dos code smell.

Refactorización: Se ha generado las funciones auxiliares mediante pinchando lo que queríamos convertir en función y poniendo Refactor => Extract Method.

4. Miembro que ha realizado la refactorización
   - Realizado por: Oiher Bernet

Refactorizacion: “Keep unit interfaces small” (createSale)

1. Código inicial

```
public Sale createSale(String title, String description, int status, float price,
Date pubDate, String sellerEmail, File file) throws FileNotUploadedException,
MustBeLaterThanTodayException, SaleAlreadyExistException {
	System.out.println(">> DataAccess: createProduct=> title= "+title+"
	seller="+sellerEmail);
	try {
		if(pubDate.before(UtilDate.trim(new Date()))) {
			throw new
			MustBeLaterThanTodayException(ResourceBundle.getBundle("Etique
			tas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
		}
		if (file==null)
		throw new
		FileNotUploadedException(ResourceBundle.getBundle("Etiquetas").getStr
		ing("DataAccess.ErrorFileNotUploadedException"));

		db.getTransaction().begin();
		Seller seller = db.find(Seller.class, sellerEmail);
		if (seller.doesSaleExist(title)) {
			db.getTransaction().commit();
			throw new
			SaleAlreadyExistException(ResourceBundle.getBundle("Etiquetas"
			).getString("DataAccess.SaleAlreadyExist"));
		}

		Sale sale = seller.addSale(title, description, status, price,
		pubDate, file);
		db.persist(seller);
		db.getTransaction().commit();
		System.out.println("sale stored "+sale+ " "+seller);

		System.out.println("hasta aqui");
		return sale;
	} catch (NullPointerException e) {
		e.printStackTrace();
```

```
            db.getTransaction().commit();
            return null;
      }
}
```

2. Código refactorizado

```
public Sale createSale(SaleCreationData saleCreationData, String sellerEmail)
throws FileNotUploadedException, MustBeLaterThanTodayException,
SaleAlreadyExistException {
      System.out.println(">> DataAccess: createProduct=> title=
      "+saleCreationData.title+" seller="+sellerEmail);
      try {
            if(saleCreationData.pubDate.before(UtilDate.trim(new Date()))) {
                  throw new
                  MustBeLaterThanTodayException(ResourceBundle.getBundle("Etique
                  tas").getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
            }
            if (saleCreationData.file==null)
            throw new
            FileNotUploadedException(ResourceBundle.getBundle("Etiquetas").getStr
            ing("DataAccess.ErrorFileNotUploadedException"));

            db.getTransaction().begin();
            Seller seller = db.find(Seller.class, sellerEmail);
            if (seller.doesSaleExist(saleCreationData.title)) {
                  db.getTransaction().commit();
                  throw new
                  SaleAlreadyExistException(ResourceBundle.getBundle("Etiquetas"
                  ).getString("DataAccess.SaleAlreadyExist"));
            }

            Sale sale = seller.addSale(saleCreationData.title,
            saleCreationData.description, saleCreationData.status,
            saleCreationData.price, saleCreationData.pubDate,
            saleCreationData.file);
            db.persist(seller);
            db.getTransaction().commit();
            System.out.println("sale stored "+sale+ " "+seller);

            System.out.println("hasta aqui");
            return sale;
      } catch (NullPointerException e) {
            e.printStackTrace();
            db.getTransaction().commit();
            return null;
      }
}
```

```
package dataAccess;

import java.io.File;
import java.util.Date;

public class SaleCreationData {
public String title;
public String description;
public int status;
public float price;
public Date pubDate;
public File file;

public SaleCreationData(String title, String description, int status, float
price, Date pubDate, File file) {
this.title = title;
this.description = description;
this.status = status;
this.price = price;
this.pubDate = pubDate;
this.file = file;
}
}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: el constructor de createSale recibía 7 parámetros, y la guía dice que como mucho 4. Con tantos parámetros del mismo tipo seguidos es fácil pasarlos en otro orden sin que el compilador se dé cuenta.

Refactorización: Se ha creado la clase SaleCreationData que agrupa los datos que describen el producto: título, descripción, estado, precio,pubData y file. Ahora el constructor de Sale recibe 2 parámetros: SaleCreationData saleCreationData y String sellerEmail. Después de esto, se ha creado la clase SaleCreationData y donde aparecía los atributos antes ponemos saleCreationData

4. Miembro que ha realizado la refactorización
    - Realizado por: Oiher Bernet

3. Julen Haro

Refactorización: "Write short units of code" (buyProduct)

1. Código inicial

```
public boolean buyProduct(String buyerEmail, Integer saleNumber) {

try {

    db.getTransaction().begin();

    Seller buyer = db.find(Seller.class, buyerEmail);

    Sale sale = db.find(Sale.class, saleNumber);

    if (buyer == null || sale == null || sale.getBuyer() != null) {

        db.getTransaction().rollback();

        return false;

    }

    float price = sale.getPrice();

    if (buyer.getMoney() < price) {

        db.getTransaction().rollback();

        return false;

    }

    buyer.addMoney(-price);

    sale.setBuyer(buyer);

    buyer.addPurchasedSale(sale);

    Bidalketa bidalketa = new Bidalketa(sale);

    sale.setBidalketa(bidalketa);

    db.persist(bidalketa);



// --- NUEVO BLOQUE MUGIMENDUAK ---

    Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new java.util.Date(), buyer);

    mugimendu.setSale(sale);

    db.persist(mugimendu);

// --------------------------------

    db.merge(sale);

    db.merge(buyer);

    db.getTransaction().commit();
```

```
    return true;
} catch (Exception e) {
    e.printStackTrace();
    db.getTransaction().rollback();
    return false;
}
}
```

2. Código refactorizado

```
public boolean buyProduct(String buyerEmail, Integer saleNumber) {
    try {
        db.getTransaction().begin();
        Seller buyer = db.find(Seller.class, buyerEmail);
        Sale sale = db.find(Sale.class, saleNumber);
        if (!canBuy(buyer, sale)) { return cancelTransaction(); }
        registerPurchase(buyer, sale);
        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        e.printStackTrace();
        return cancelTransaction();
    }
}
private boolean canBuy(Seller buyer, Sale sale) {
    return buyer != null && sale != null && sale.getBuyer() == null && buyer.getMoney()
>= sale.getPrice();
}
private void registerPurchase(Seller buyer, Sale sale) {
    buyer.addMoney(-sale.getPrice());
    sale.setBuyer(buyer);
    buyer.addPurchasedSale(sale);
```

```
    Bidalketa bidalketa = new Bidalketa(sale);

    sale.setBidalketa(bidalketa);

    db.persist(bidalketa);

    Mugimenduak mugimendu = new Mugimenduak("EROSKETA", new java.util.Date(), buyer);

    mugimendu.setSale(sale);

    db.persist(mugimendu);

    db.merge(sale);

    db.merge(buyer);

}

private boolean cancelTransaction() {

    db.getTransaction().rollback();

    return false;

}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: El método buyProduct de DataAccess tenía 33 líneas de código, más del doble del límite de 15 líneas. En un mismo método se comprobaba si se podía hacer la compra, se actualizaban el comprador y el producto, se creaba el envío y el movimiento y se gestionaba la transacción, por lo que era difícil de leer y de probar por partes.

Refactorización: con Refactor > Extract Method de Eclipse he sacado tres métodos privados: canBuy, registerPurchase y cancelTransaction. Ahora buyProduct tiene 14 líneas y solo se encarga de la transacción. El comportamiento es el mismo: los 28 tests de buyProduct (BD y Mock) siguen pasando.

4. Miembro que ha realizado la refactorización
    - Realizado por: Julen Haro

Refactorización: "Write simple units of code" (resolveReport)

1. Código inicial

```
public boolean resolveReport(Integer saleNumber, Salaketa salaketa,boolean aceptar) {
```

```
    try {
        db.getTransaction().begin();
        Sale sale = db.find(Sale.class, saleNumber);
        if (sale == null || salaketa == null) {
            db.getTransaction().rollback();
            return false;
        }
        sale.getSalaketak().removeIf(s -> Objects.equals(s.getId(), salaketa.getId()));
        Salaketa s = db.find(Salaketa.class, salaketa.getId());
        if (s != null) {
            s.setTratatuta(true);
            db.merge(s);
        }
        if (aceptar) {
            Seller owner = sale.getSeller();
            if (owner != null) {
                owner.removeSale(sale);
                db.merge(owner);
            }
            db.remove(sale);
        } else {
            db.merge(sale);
        }
        db.getTransaction().commit();
        return true;
    } catch (Exception e) {
        e.printStackTrace();
        db.getTransaction().rollback();
        return false;
    }
```

```
}
```

2. Código refactorizado

```
public boolean resolveReport(Integer saleNumber, Salaketa salaketa,boolean aceptar) {

    try {

        db.getTransaction().begin();

        Sale sale = db.find(Sale.class, saleNumber);

        if (sale == null || salaketa == null) {

            return cancelTransaction();

        }

        markReportAsSolved(sale, salaketa);

        if (aceptar) {

            deleteReportedSale(sale);

        } else {

            db.merge(sale);

        }

        db.getTransaction().commit();

        return true;

    } catch (Exception e) {

        e.printStackTrace();

        return cancelTransaction();

    }

}

private void markReportAsSolved(Sale sale, Salaketa salaketa) {

        sale.getSalaketak().removeIf(s -> Objects.equals(s.getId(), salaketa.getId()));

        Salaketa s = db.find(Salaketa.class, salaketa.getId());

        if (s != null) {

            s.setTratatuta(true);

            db.merge(s);

        }

}
```

```
private void deleteReportedSale(Sale sale) {

    Seller owner = sale.getSeller();

    if (owner != null) {

        owner.removeSale(sale);

        db.merge(owner);

    }

    db.remove(sale);

}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: resolveReport tenía 6 puntos de decisión, por encima del máximo de 4, con ifs anidados dentro del if de aceptar.

Refactorización: con Extract Method he separado las dos tareas que hacía dentro: markReportAsSolved y deleteReportedSale. resolveReport se queda con 4 puntos de decisión y cada método nuevo tiene solo uno. También reutiliza cancelTransaction para los rollback.

4. Miembro que ha realizado la refactorización
    - Realizado por: Julen Haro

Refactorización: "Duplicate code" (getSales y getPublishedSales)

1. Código inicial

```
public List<Sale> getSales(String desc) {

    System.out.println(">> DataAccess: getProducts=> from= "+desc);

    List<Sale> res = new ArrayList<Sale>();

    TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE ?1
AND s.buyer IS NULL", Sale.class);

    query.setParameter(1, "%"+desc+"%");

    List<Sale> sales = query.getResultList();

    for (Sale sale:sales){

        res.add(sale);

    }

    return res;
```

```
}

public List<Sale> getPublishedSales(String desc, Date pubDate) {

    System.out.println(">> DataAccess: getProducts=> from= "+desc);

    List<Sale> res = new ArrayList<Sale>();

    TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE ?1
AND s.pubDate <=?2 AND s.buyer IS NULL",Sale.class);

    query.setParameter(1, "%"+desc+"%");

    query.setParameter(2,pubDate);

    List<Sale> sales = query.getResultList();

    for (Sale sale:sales){

        res.add(sale);

    }

    return res;

}
```

2. Código refactorizado

```
public List<Sale> getSales(String desc) {

    TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE ?1
AND s.buyer IS NULL", Sale.class);

    return getSalesByDescription(query, desc);

}

public List<Sale> getPublishedSales(String desc, Date pubDate) {

    TypedQuery<Sale> query = db.createQuery("SELECT s FROM Sale s WHERE s.title LIKE ?1
AND s.pubDate <=?2 AND s.buyer IS NULL",Sale.class);

    query.setParameter(2,pubDate);

    return getSalesByDescription(query, desc);

}

private List<Sale> getSalesByDescription(TypedQuery<Sale> query, String desc) {

    System.out.println(">> DataAccess: getProducts=> from= "+desc);

    query.setParameter(1, "%"+desc+"%");

    return new ArrayList<>(query.getResultList());

}
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: getSales y getPublishedSales eran casi iguales: los dos escribían el mismo mensaje por consola, ponían el mismo parámetro de la descripción en la consulta y copiaban el resultado a una lista nueva con el mismo bucle for. Solo cambiaba la consulta y el parámetro de la fecha.

Refactorización: he sacado la parte común a un método privado getSalesByDescription, que recibe la consulta y la descripción, pone el parámetro y devuelve la lista. Además el bucle for se ha cambiado por new ArrayList<>(query.getResultList()). Cada método público solo crea su consulta y llama al método común.

4. Miembro que ha realizado la refactorización
    - Realizado por: Julen Haro

Refactorización: "Keep unit interfaces small" (constructor de Sale)

1. Código inicial

```
// Sale

public Sale(String title, String description, int status, float price, Date pubDate,
File file, Seller seller) {

    super();

    this.title = title;

    this.description = description;

    this.status = status;

    this.price = price;

    this.pubDate = pubDate;

    if (file != null) {

        this.fileName = file.getName();

        try {

            BufferedImage img1 = ImageIO.read(file);

            String path = "src/main/resources/images/";

            File outputfile = new File(path + file.getName());

            ImageIO.write(img1, "png", outputfile);

        } catch(IOException ex) {

            System.out.println("Error guardando imagen: " + ex.getMessage());
```

```
        }
    }
    this.seller = seller;
}
// Seller.addSale
Sale sale=new Sale(title, description, status, price, pubDate, file, this);
```

2. Código refactorizado

```
package domain;
public class ProductInfo {
    private String title;
    private String description;
    private int status;
    private float price;
    public ProductInfo(String title, String description, int status, float price) {
        this.title = title;
        this.description = description;
        this.status = status;
        this.price = price;
    }
    public String getTitle() {
        return title;
    }
    public String getDescription() {
        return description;
    }
    public int getStatus() {
        return status;
    }
    public float getPrice() {
```

```
        return price;
    }
}
// Sale
public Sale(ProductInfo info, Date pubDate, File file, Seller seller) {
    super();
    this.title = info.getTitle();
    this.description = info.getDescription();
    this.status = info.getStatus();
    this.price = info.getPrice();
    this.pubDate = pubDate;
    if (file != null) {
        this.fileName = file.getName();
        try {
            BufferedImage img1 = ImageIO.read(file);
            String path = "src/main/resources/images/";
            File outputfile = new File(path + file.getName());
            ImageIO.write(img1, "png", outputfile);
        } catch(IOException ex) {
            System.out.println("Error guardando imagen: " + ex.getMessage());
        }
    }
    this.seller = seller;
}
// Seller.addSale
Sale sale=new Sale(new ProductInfo(title, description, status, price), pubDate, file,
this);
```

3. Descripción del code smell detectado y descripción de la refactorización realizada

Code Smell: el constructor de Sale recibía 7 parámetros, y la guía dice que como mucho 4. Con tantos parámetros del mismo tipo seguidos es fácil pasarlos en otro orden sin que el compilador se dé cuenta.

Refactorización: he creado la clase ProductInfo que agrupa los datos que describen el producto: título, descripción, estado y precio. Ahora el constructor de Sale recibe 4 parámetros: ProductInfo info, Date pubDate, File file y Seller seller, y saca los datos con los getters. He actualizado las llamadas al constructor. El resto del código sigue llamando a addSale igual, así que no cambia nada más y todos los tests siguen pasando.

4. Miembro que ha realizado la refactorización
    - Realizado por: Julen Haro